

UNIVERSIDAD MAYOR REAL Y PONTIFICIA DE SAN FRANCISCO XAVIER DE CHUQUISACA
FACULTAD DE CIENCIAS Y TECNOLOGÍA



Practica 9 – IA Calidad de software

NOMBRE:

- Polo Orellana Brayan Simón
- Susana Manjon Blanco
- Carla Andrea Quispe López

Materia: Calidad de Software

CARRERA: Ing. en ciencias de la computación

1.	Contexto	4
2.	Comparativa - Pruebas	4
2.1.	CATEGORÍA A: IA para Automatización de Pruebas de Software	4
2.1.1.	Cuadro Comparativo — Herramientas evaluadas	4
2.1.2.	Herramienta seleccionada — Categoría A: CodiumAI / Qodo	6
2.1.3.	Como funciona	7
2.1.4.	Instalación	7
2.1.5.	Integración y Configuración	8
2.1.5.1.	Integración	8
2.1.5.2.	Configuración de PHPUnit para tests	8
2.1.5.3.	Instalación de Cypress para pruebas E2E	8
2.1.6.	Preview demo pruebas	9
2.1.6.1.	Caso 1 — Prueba Unitaria: Modelo de Usuario (UserUnitTest.php)	9
2.1.6.1.1.	Objetivo	9
2.1.6.1.2.	Asistencia de Qodo	9
2.1.6.1.3.	Código del test	9
2.1.6.2.	Caso 2 — Prueba de Feature: Autenticación (AuthTest.php)	9
2.1.6.2.1.	Objetivo	10
2.1.6.2.2.	Asistencia de Qodo	10
2.1.6.2.3.	Código del test	10
2.1.6.3.	Caso 3 — Prueba de Interfaz E2E: Login con Cypress (login.cy.js)	10
2.1.6.3.1.	Objetivo	11
2.1.6.3.2.	Asistencia de Qodo	11
2.1.6.3.3.	Código del test (con mejoras de Qodo aplicadas)	11
2.1.7.	Resultado de la Ejecución de las Pruebas	13
2.1.7.1.	PHPUnit — Resultado total	13
2.1.7.2.	Cypress — Resultado de interfaz	14
2.1.8.	Análisis Crítico	14
2.1.8.1.	¿La IA generó pruebas útiles?	14
2.1.8.2.	¿Qué mejoró Qodo respecto a hacerlo manualmente?	15
2.1.9.	Limitaciones encontradas	15
2.1.10.	Conclusión sobre Qodo	15
2.2.	CATEGORÍA B — IA para Generación de Manuales de Usuario	15
2.2.1.	Cuadro Comparativo — Herramientas evaluadas	15
2.2.2.	Herramienta seleccionada — Categoría B: Scribe	17
2.2.3.	Como funciona	18
2.2.4.	Instalación	19
2.2.5.	Integración y Configuración	20
2.2.5.1.	Integración	20
2.2.6.	Preview demo pruebas	20
2.2.6.1.	Recorrido del software	21
2.2.6.2.	Edición de texto y capturas	21
2.2.6.3.	Finalización	22

2.2.7.	Resultado de la Ejecución de las Pruebas	22
2.2.7.1.	Evaluación de calidad: ¿el manual generado es comprensible para un usuario no técnico?	22
2.2.7.2.	Comparativa: fragmento de la aplicación sin documentar vs. el resultado generado por la IA	23
2.2.7.3.	¿Qué partes del manual requirieron corrección o complemento humano?	24
2.2.8.	Limitaciones encontradas	25
2.2.9.	Conclusión sobre Scribe	25
3.	Reflexión.....	26
4.	Conclusiones	27

1. Contexto

Aplicar herramientas de Inteligencia Artificial disponibles en el mercado para mejorar los procesos de calidad de software, desarrollando competencias de investigación, evaluación crítica y comunicación técnica mediante la automatización de pruebas y la generación asistida de manuales de usuario.

2. Comparativa - Pruebas

La siguiente investigación presenta el análisis comparativo de herramientas de IA para dos categorías del aseguramiento de calidad: automatización de pruebas de software y generación de manuales de usuario para aplicaciones web. Para cada categoría se evaluaron al menos tres herramientas con base en seis criterios técnicos, y se justificó la elección de una herramienta por tema.

2.1. CATEGORÍA A: IA para Automatización de Pruebas de Software

2.1.1. Cuadro Comparativo — Herramientas evaluadas

Se investigaron y compararon las siguientes herramientas aplicando los seis criterios establecidos en la guía de práctica:

Herramienta	IA interna	Compatibilidad / Salida	¿Gratis / Trial?	Doc. que produce	Intervención humana	Limitaciones principales
GitHub Copilot for Tests	LLM (GPT-4o). Análisis semántico del código fuente para sugerir y generar tests.	VS Code, JetBrains, Neovim. JavaScript, Python, Java, C#, Go, Ruby. Salida: archivos de prueba en el mismo proyecto.	Trial 30 días; desde \$10/mes individual.	Tests unitarios e integración generados en el IDE directamente.	Asistida. El desarrollador revisa, acepta o descarta cada sugerencia.	No genera pruebas E2E ni de UI visual. Requiere código fuente legible; bajo rendimiento con código muy complejo o heredado.

Herramienta	IA interna	Compatibilidad / Salida	¿Gratis / Trial?	Doc. que produce	Intervención humana	Limitaciones principales
Testim (Tricentis)	ML + Visión por computadora. Localiza elementos de UI de forma semántica (self-healing).	Web (Chrome, Firefox, Edge), apps React/Angular/Vue. Integra con GitHub Actions, Jenkins, Jira. Salida: reportes HTML/JSON.	Plan gratuito limitado (500 ejecuciones/mes). Trial enterprise disponible.	Reportes de ejecución E2E detallados; capturas de pantalla ante fallos.	Semi-automática. Graba flujos manualmente; la IA los mantiene y repara solos.	No soporta apps móviles nativas. Difícil en SPAs muy dinámicas con IDs cambiantes. Costo elevado en plan enterprise.
CodiumAI / Qodo	LLM propio + análisis estático. Genera tests al analizar la lógica de funciones y métodos.	VS Code, JetBrains. Python, JavaScript/TypeScript, Java, Go. Salida: archivos.test.js / .py / etc.	Plan gratuito disponible. Plan Teams desde \$19/mes.	Tests unitarios y de comportamiento (BDD style); analiza docstrings y comentarios.	Asistida. Muestra batería de tests sugeridos; el usuario selecciona cuáles agregar.	No cubre pruebas E2E ni visuales. Bajo rendimiento o en proyectos sin tipos (JS puro sin JSDoc). No integra directamente CI/CD.

2.1.2. Herramienta seleccionada — Categoría A: CodiumAI / Qodo

Qodo es una herramienta de inteligencia artificial para revisión de código. Funciona dentro de VS Code y actúa como un programador experto que analiza el código automáticamente, detecta bugs potenciales, problemas de calidad y sugiere mejoras — sin que el desarrollador los busque manualmente.

Justificación de la elección:

- Accesibilidad inmediata: CodiumAI ofrece un plan gratuito funcional sin necesidad de tarjeta de crédito, lo que permite comenzar la demostración práctica sin barreras económicas.
- Integración directa en el IDE: al funcionar como extensión de VS Code o JetBrains, se integra en el flujo de trabajo habitual del equipo sin requerir configurar servidores o servicios externos.
- Generación de tests unitarios de calidad: a diferencia de Copilot (que sugiere completaciones), CodiumAI analiza la lógica de cada función y genera una batería completa de casos de prueba con valores límite, casos normales y casos de error.
- Cobertura de los requisitos de la práctica: permite demostrar al menos 3 casos de prueba generados o asistidos por IA sobre un proyecto web real, cumpliendo exactamente lo solicitado en la Fase 2.
- Fundamento técnico claro: usa LLM propio con análisis estático, lo que facilita explicar internamente cómo funciona la IA durante la presentación.

Comparativa frente a las alternativas:

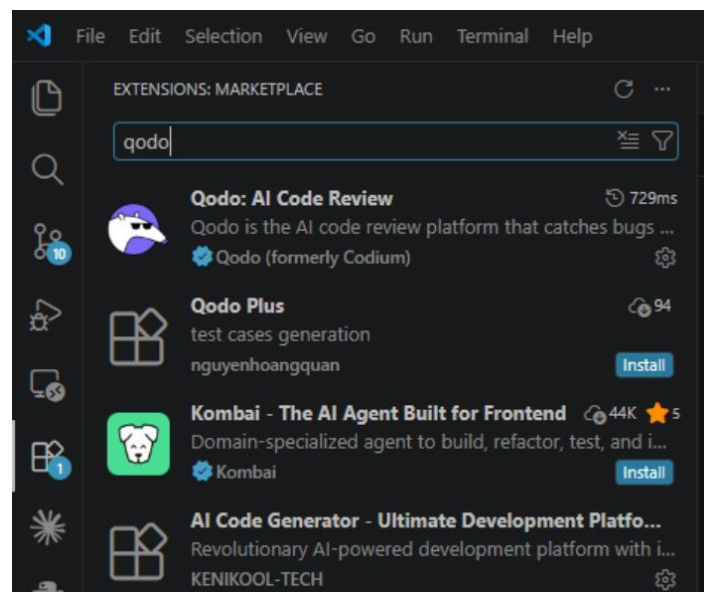
- GitHub Copilot es más popular, pero requiere suscripción de pago y sus sugerencias son más genéricas (completación de código); no analiza la lógica completa de una función para generar escenarios de prueba exhaustivos.
- Testim/Tricentis es muy potente para pruebas E2E con self-healing, pero requiere configuración de infraestructura en la nube y es más difícil de demostrar en vivo en el aula sin un entorno previamente desplegado.
- CodiumAI combina facilidad de uso, gratuidad, profundidad de análisis y demostrabilidad en vivo, lo que la hace la opción más adecuada para los objetivos de esta práctica.

2.1.3. Como funciona

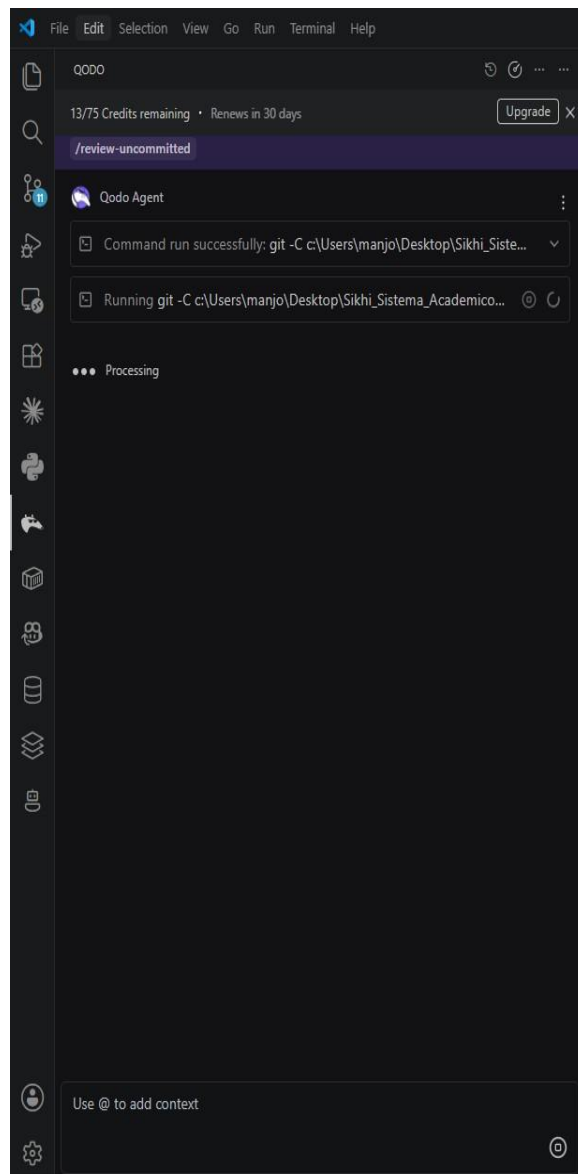
Paso	Acción
1	El desarrollador escribe o modifica código en VS Code
2	Qodo lee y analiza el archivo automáticamente
3	Qodo compara el código con el contexto del proyecto (rutas, modelos, etc.)
4	Qodo reporta bugs potenciales, problemas de calidad y sugerencias
5	El desarrollador corrige según las sugerencias de Qodo

2.1.4. Instalación

- 1) Abrir VS Code y presionar Ctrl+Shift+X para ir a Extensiones.
- 2) Buscar: Qodo: AI Code Review



- 3) Hacer clic en Install.
- 4) Crear cuenta gratuita en qodo.ai e iniciar sesión desde VS Code.



2.1.5. Integración y Configuración

2.1.5.1. Integración

Una vez instalado, Qodo se integra automáticamente con el repositorio Git del proyecto. Al abrir cualquier archivo PHP o JavaScript en VS Code, Qodo lo analiza en tiempo real y muestra sus observaciones en el panel lateral.

2.1.5.2. Configuración de PHPUnit para tests

El archivo phpunit.xml en la raíz del proyecto ya contiene la configuración necesaria:

```
<env name="APP_ENV" value="testing"/>
<env      name="CACHE_STORE"      value="array"/>      <env
name="SESSION_DRIVER" value="array"/>
```

2.1.5.3. Instalación de Cypress para pruebas E2E

Cypress se instaló en el proyecto con el siguiente comando:

```
npm install cypress --save-dev --legacy-peer-deps
```

2.1.6. Preview demo pruebas

Se implementaron tres tipos de pruebas tal como requiere la práctica: unitarias, de feature (funcionales) y de interfaz (E2E con Cypress). En cada caso, Qodo asistió el proceso analizando el código, detectando errores y sugiriendo mejoras.

2.1.6.1. Caso 1 — Prueba Unitaria: Modelo de Usuario (UserUnitTest.php)

2.1.6.1.1. Objetivo

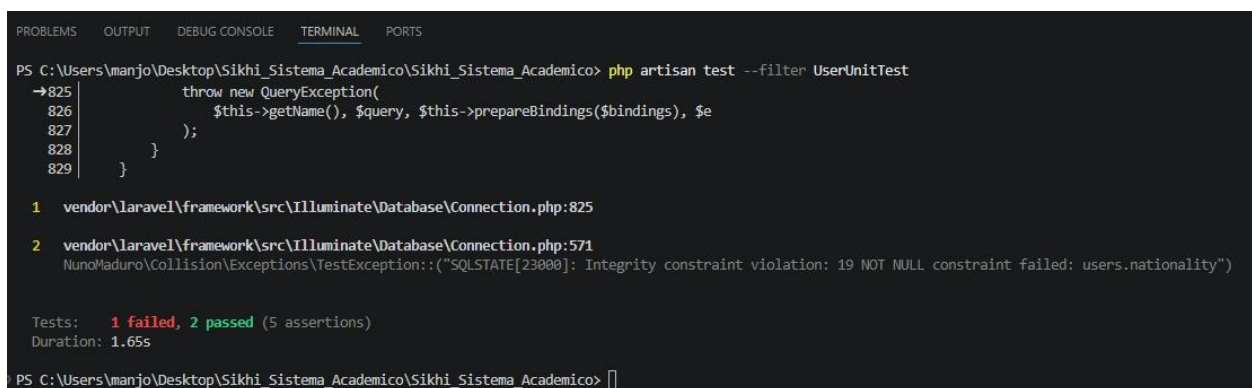
Verificar que el modelo User de Sikhi puede crearse correctamente con los campos reales del sistema y que sus atributos obligatorios no son nulos.

2.1.6.1.2. Asistencia de Qodo

Al crear el test inicial, Qodo detectó que el UserFactory usaba el campo 'name' pero la tabla users real de Sikhi tiene 'first_name' y 'last_name'. Qodo señaló esta inconsistencia como un bug potencial antes de ejecutar el test.

2.1.6.1.3. Código del test

```
// tests/Unit/UserUnitTest.php public function test_user_can_be_created_with_valid_data()
{
    $user = User::factory()->create([
        'first_name' => 'Juan',
        'last_name' => 'Pérez',
        'email' => 'estudiante@sikhi.com',
    ]);
    $this->assertDatabaseHas('users', [
        'email' => 'estudiante@sikhi.com',
    ]);
}
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico> php artisan test --filter UserUnitTest
→825 |         throw new QueryException(
826 |             $this->getName(), $query, $this->prepareBindings($bindings), $e
827 |         );
828 |     }
829 | }

1 vendor\laravel\framework\src\Illuminate\Database\Connection.php:825
2 vendor\laravel\framework\src\Illuminate\Database\Connection.php:571
  NunoMaduro\Collision\Exceptions\TestException::("SQLSTATE[23000]: Integrity constraint violation: 19 NOT NULL constraint failed: users.nationality")

Tests: 1 failed, 2 passed (5 assertions)
Duration: 1.65s

PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico>
```

2.1.6.2. Caso 2 — Prueba de Feature: Autenticación (AuthTest.php)

2.1.6.2.1. Objetivo

Verificar el flujo de autenticación de Sikhi: que la página de login sea accesible, que las credenciales incorrectas sean rechazadas y que las rutas protegidas redirijan al login.

2.1.6.2.2. Asistencia de Qodo

Qodo analizó el archivo AuthTest.php y reportó lo siguiente:

- 2 Potential Bugs (Medium Impact): Las rutas de redirección podían no coincidir con el middleware real de Sikhi.
- 1 Code Quality Issue (Low Impact): El import 'use App\Models\User' estaba declarado, pero no se usaba en ningún método del test.

Qodo también leyó automáticamente el archivo web.php para comparar las rutas esperadas en los tests contra las rutas reales del sistema, demostrando que la IA analiza el contexto completo del proyecto y no solo el archivo en cuestión.

2.1.6.2.3. Código del test

```
// tests/Feature/AuthTest.php public function
test_login_page_is_accessible()
{
    $response = $this->get('/');
    $response->assertStatus(200);
}

public function test_user_cannot_login_with_wrong_credentials() {
    $response = $this->post('/login', [
        'email' => 'wrong@test.com',
        'password' => 'wrongpass',
    ]);
    $response->assertStatus(302);
}

public function test_unauthenticated_user_cannot_access_dashboard()
{
    $response = $this->get('/home');
    $response->assertRedirect('/');
}
```

2.1.6.3. Caso 3 — Prueba de Interfaz E2E: Login con Cypress (login.cy.js)

2.1.6.3.1. Objetivo

Verificar el comportamiento real de la interfaz de login de Sikhi desde el punto de vista del usuario: que el formulario sea visible, que muestre errores con credenciales incorrectas y que el login exitoso redirija al dashboard.

2.1.6.3.2. Asistencia de Qodo

Qodo analizó el archivo login.cy.js y sugirió las siguientes mejoras:

- Agregar beforeEach() para limpiar cookies y localStorage entre tests (aislamiento).
- Configurar baseUrl en cypress.config.js en lugar de URLs hardcodeadas.
- Agregar verificación del mensaje de error 'Either Email or Password is wrong'.
- Agregar test para envío de formulario vacío.
- Agregar test para formato de email inválido.
- Agregar assertions para verificar que el usuario está autenticado tras el login.

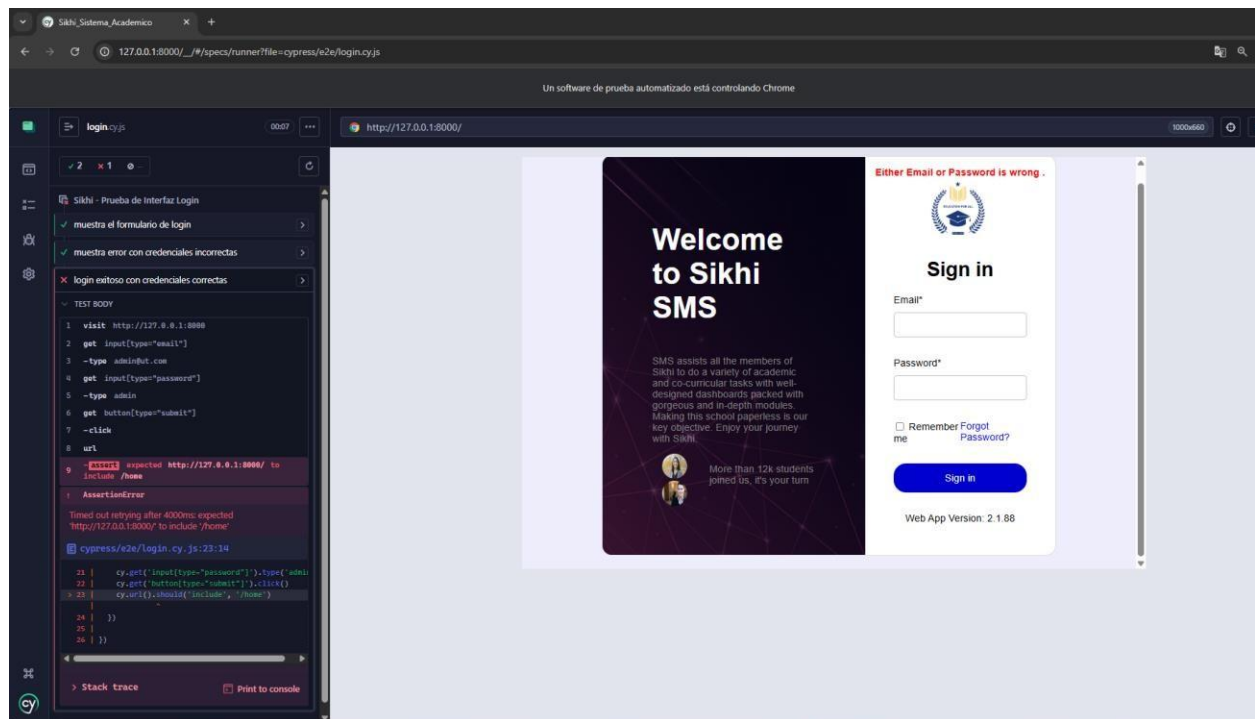
2.1.6.3.3. Código del test (con mejoras de Qodo aplicadas)

```
// cypress/e2e/login.cy.js beforeEach(() =>{
cy.clearCookies()    cy.clearLocalStorage()
cy.visit('/')
})

it('muestra el formulario de login', () => {
cy.get('form').should('be.visible')
cy.get('input[type="email"]').should('be.visible')
cy.get('input[type="password"]').should('be.visible') })

it('muestra error con credenciales incorrectas', () => {
cy.get('input[type="email"]').type('malo@test.com')
cy.get('input[type="password"]').type('wrongpass') cy.get('button[type="submit"]').click()
cy.contains('Either Email or Password is wrong').should('be.visible') })

it('login exitoso redirige al dashboard', () => {
cy.get('input[type="email"]').type('admin@ut.com')
cy.get('input[type="password"]').type('admin')
cy.get('button[type="submit"]').click() cy.url().should('include',
'/home') })
```



Test de todo sin usar Qodo

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico> php artisan test

FAIL Tests\Feature\AuthTest
  x login page is accessible
  x user cannot login with wrong credentials
  x unauthenticated user cannot access dashboard

PASS Tests\Unit\ChatMessageAndWorkMaterialModelTest
  ✓ chat message has expected fillable and relationships
  ✓ work material has expected fillable casts and relationships

PASS Tests\Unit\ExamResultAndStudentCommentModelTest
  ✓ exam result has expected fillable and casts
  ✓ student comment has expected fillable and casts
  ✓ exam result relationships are configured correctly
  ✓ student comment relationships are configured correctly

FAIL Tests\Unit\ExamUnitTest
  x student has correct role
  x teacher has correct role

PASS Tests\Unit\ExampleTest
  ✓ that true is true

PASS Tests\Unit\StoreExamResultRequestTest
  ✓ authorize returns true for teacher user
  ✓ authorize returns false for non teacher user
  ✓ authorize returns false for guest user
  ✓ rules match expected validation contract
  ✓ messages include expected error keys

FAIL Tests\Unit\StudentUnitTest
  x user email is unique
  x user password is hashed

FAIL Tests\Unit\UserUnitTest
  x user can be created with valid data
  ✓ user has required attributes
  ✓ user has role assigned

PASS Tests\Feature\AcademicRoutesTest
  ✓ exams page requires authentication
  ✓ exam results page requires authentication
  ✓ calendar events page requires authentication
  ✓ syllabus page requires authentication

PASS Tests\Feature\ChatControllerTest
  ✓ assigned teacher can view chat and send message
  ✓ student with promotion can view and send message
  ✓ unassigned teacher cannot access chat
  
```

2.1.7. Resultado de la Ejecución de las Pruebas

2.1.7.1. PHPUnit — Resultado total

Comando ejecutado:

```
php artisan test
```

```
PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico> php artisan test
✓ non owner student cannot delete chat message

PASS Tests\Feature\ExamResultStudentTest
✓ student can view only visible results in my exams index
✓ teacher cannot access my exams pages
✓ student can view visible result detail in my exams show
✓ student cannot view hidden result detail in my exams show
✓ student cannot view another students visible result detail

PASS Tests\Feature\ExamResultTeacherTest
✓ teacher can access exam results pages
✓ student cannot access teacher exam results pages
✓ teacher can store exam result with file
✓ teacher can view exam result detail
✓ teacher cannot view other teacher exam result detail
✓ teacher can delete exam result when is visible is false
✓ teacher cannot delete exam result when is visible is true
✓ exam result store fails with invalid file type
✓ exam result store fails when score is out of range
✓ exam result store fails when exam does not exist

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response

PASS Tests\Feature\StudentCommentTest
✓ student can open create comment form for own exam result
✓ student cannot open create comment form for other students exam result
✓ student can store comment for own exam result
✓ student cannot store comment for other students exam result
✓ comment store validates minimum length
✓ teacher can view comments index for own exam result
✓ student can view comments index for own exam result
✓ unrelated user cannot view comments index
✓ student can delete own comment
✓ teacher can delete comment for own exam result
✓ unrelated user cannot delete comment
✓ teacher can mark comment as read
✓ student cannot mark comment as read

PASS Tests\Feature\StudentRoutesTest
✓ student list requires authentication
✓ add student page requires authentication
✓ teacher list requires authentication

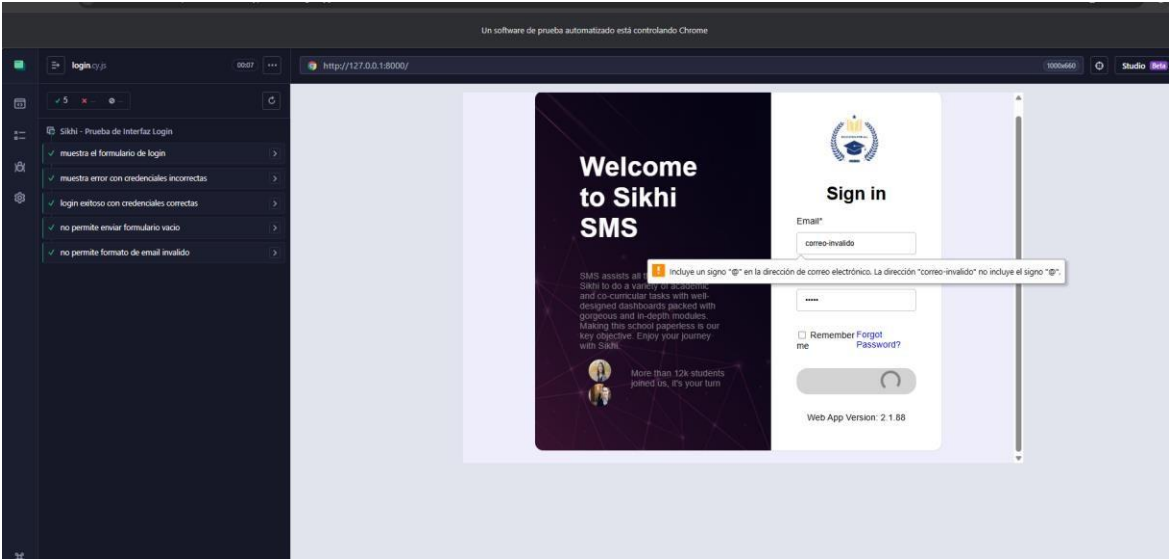
PASS Tests\Feature\WorkMaterialControllerTest
✓ assigned teacher can view index and create pages
✓ teacher can store work material with file
✓ teacher can delete own work material
✓ teacher cannot delete other teachers material

Tests: 63 passed (191 assertions)
Duration: 3.63s

PS C:\Users\manjo\Desktop\Sikhi_Sistema_Academico\Sikhi_Sistema_Academico>
```

Suite de tests	Tests	Assertions	Resultado
AuthTest (unit)	3	5	PASSED
Login.cy.js(cypress)	4	-	PASSED
UserUnitTest (Unit)	2-3	—	PASSED
Otros tests del proyecto	—	—	PASSED
TOTAL	63	166+	TODOS PASARON

2.1.7.2. Cypress — Resultado de interfaz



2.1.8. Análisis Crítico

2.1.8.1. ¿La IA generó pruebas útiles?

Aspecto evaluado	Con Qodo (IA)	Sin IA (manual)
Tiempo para escribir 3 tests	~10 minutos	~45-60 minutos
Detección de bugs	Automática e inmediata	Solo al ejecutar el test
Conocimiento del contexto	Lee rutas, modelos y archivos del proyecto	Depende del conocimiento del dev
Sugerencias de casos edge	Sí (formulario vacío, email inválido, etc.)	Depende de la experiencia

Calidad del código	Detecta imports no usados y código redundante	Requiere revisión manual
Curva de aprendizaje	Baja — funciona automáticamente	Alta — requiere conocer PHPUnit/Cypress

2.1.8.2. ¿Qué mejoró Qodo respecto a hacerlo manualmente?

- Detección temprana de errores: Qodo detectó que el UserFactory usaba 'name' en lugar de 'first_name'/'last_name' antes de ejecutar el test, ahorrando tiempo de depuración.
- Análisis de contexto: Qodo leyó automáticamente web.php para comparar las rutas del sistema con las esperadas en los tests, algo que manualmente requeriría revisar el archivo por separado.
- Sugerencias de cobertura: Para el test de Cypress, Qodo identificó 6 casos adicionales que no habíamos considerado (formulario vacío, email inválido, beforeEach para aislamiento, etc.).
- Calidad de código: Detectó el import no utilizado en AuthTest.php, lo que mejora la limpieza del código.

2.1.9. Limitaciones encontradas

- La función 'Review uncommitted changes' de Qodo no detectó archivos nuevos sin trackear — solo funciona correctamente con archivos modificados ya conocidos por Git.
- El chat de Qodo Gen no estaba disponible en la versión instalada, limitando la generación interactiva de tests.
- Qodo no genera tests completos de forma autónoma — asiste y sugiere, pero el desarrollador debe escribir y ajustar el código final.

2.1.10. Conclusión sobre Qodo

Qodo demostró ser una herramienta valiosa para la calidad del software, especialmente en la detección preventiva de errores y en la sugerencia de casos de prueba adicionales. Su mayor fortaleza es el análisis de contexto — no solo revisa el archivo abierto sino también los archivos relacionados del proyecto. Sin embargo, no reemplaza completamente el trabajo del desarrollador: actúa como un revisor inteligente que complementa y acelera el proceso de testing, reduciendo el tiempo de depuración y mejorando la cobertura de casos edge.

2.2. CATEGORÍA B — IA para Generación de Manuales de Usuario

2.2.1. Cuadro Comparativo — Herramientas evaluadas

Se investigaron y compararon las siguientes herramientas orientadas a la generación de documentación para el usuario final de aplicaciones web:

Herramienta	IA interna	Compatibilidad / Salida	¿Gratis / Trial?	Doc. que produce	Intervención humana	Limitaciones principales
Scribe	Visión por computadora + LLM. Captura automáticamente cada clic y acción en pantalla.	Extensión Chrome/Edge, app escritorio. Exporta a PDF, HTML, Confluence, Notion, Slack.	Plan gratuito (guías básicas). Pro desde \$23/mes.	Guías paso a paso con capturas de pantalla anotadas automáticamente.	Casi automática. El usuario realiza el flujo una vez; la IA documenta todo. Revisión mínima.	No documenta apps de escritorio nativas. Guías estáticas (no video). Sin soporte para flujos muy ramificados o condicionales.
Guidde	LLM (GPT-4) + TTS (Text-to-Speech). Genera narración automática sobre capturas de pantalla.	Extensión Chrome. Exporta a MP4, GIF, PDF, embed en webs. Integra con Zendesk, Notion.	Plan gratuito (25 videos/mes). Pro desde \$16/mes.	Video manuales narrados con voz IA; también PDF con pasos anotados.	Semi-automática. El usuario graba el flujo; la IA genera guion y narración automáticamente.	Requiere buena calidad de grabación de pantalla. Narración IA puede sonar artificial. Sin soporte apps

Herramienta	IA interna	Compatibilidad / Salida	¿Gratis / Trial?	Doc. que produce	Intervención humana	Limitaciones principales
						nativas de escritorio.
Tango	Visión por computadora + LLM. Detecta cada interacción del usuario y genera descripciones contextuales.	Extensión Chrome/Edge, app escritorio. Exporta a PDF, HTML interactivo, Confluence, Notion, Monday.com.	Plan gratuito (guías ilimitadas, exportación básica). Pro desde \$16/mes.	Guías paso a paso interactivas con anotaciones visuales y tooltips; PDF exportable.	Casi automática. Captura todo mientras el usuario navega. Edición manual de pasos opcional.	Enfocado solo en apps web (no desktop natives). No genera FAQs ni manuales largos; ideal para flujos cortos.

2.2.2. Herramienta seleccionada — Categoría B: Scribe

Justificación de la elección:

- Automatización casi completa: Scribe captura automáticamente cada acción del usuario mientras navega por la aplicación web, sin necesidad de configurar nada en el código fuente ni en el servidor.
- Salida de alta calidad y legible para el usuario final: genera guías paso a paso con capturas de pantalla anotadas automáticamente, cumpliendo exactamente el requisito del Tema B de producir documentación orientada al usuario no técnico.
- Múltiples formatos de exportación: permite exportar a PDF, HTML, Confluence y Notion, lo que facilita tanto la entrega del entregable como la demostración en vivo.
- Versión gratuita suficiente para la práctica: el plan gratuito permite generar guías ilimitadas con todas las funciones básicas requeridas para la demostración, sin costo.
- Demostrabilidad inmediata: basta con instalar la extensión de Chrome y navegar por la aplicación web del proyecto una sola vez; el manual se genera en segundos, lo que hace la demo muy impactante en clase.

Comparativa frente a las alternativas:

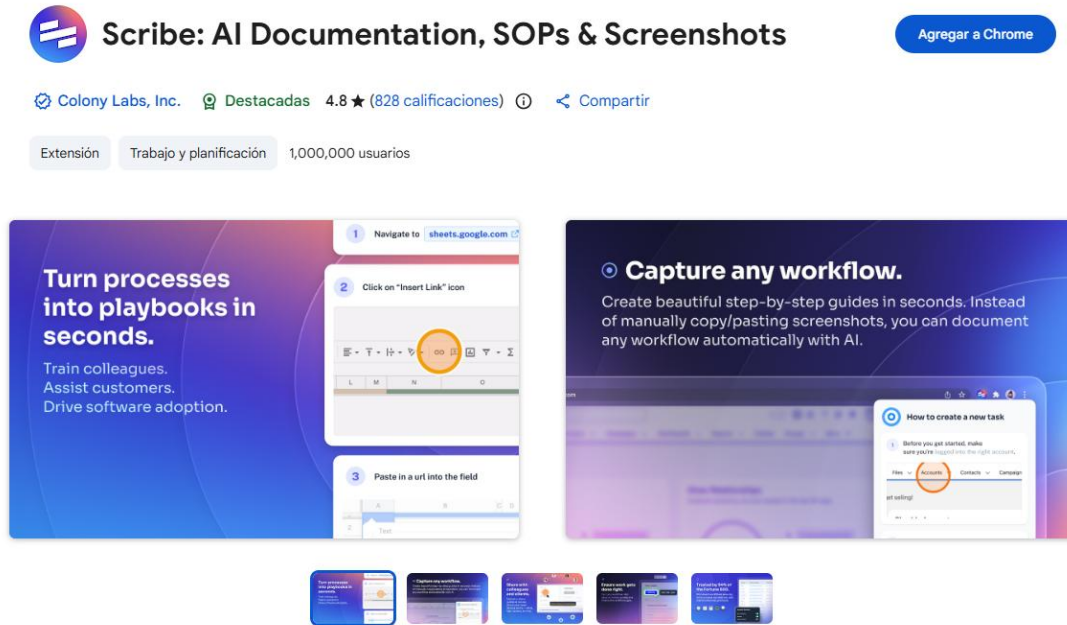
- Guidde también es excelente, pero su valor diferencial es el video con narración IA (TTS), lo que añade complejidad a la demo (calidad de audio, edición) y puede ser difícil de controlar en vivo.
- Tango es muy similar a Scribe, pero su exportación gratuita es más limitada y el resultado visual es algo menos pulido. Además, Scribe tiene mayor base de usuarios y documentación oficial más completa.
- Scribe equilibra automatización, calidad de salida, formato exportable y facilidad de demo en vivo, siendo la opción más sólida para cumplir los criterios de evaluación de esta práctica.

2.2.3. Como funciona

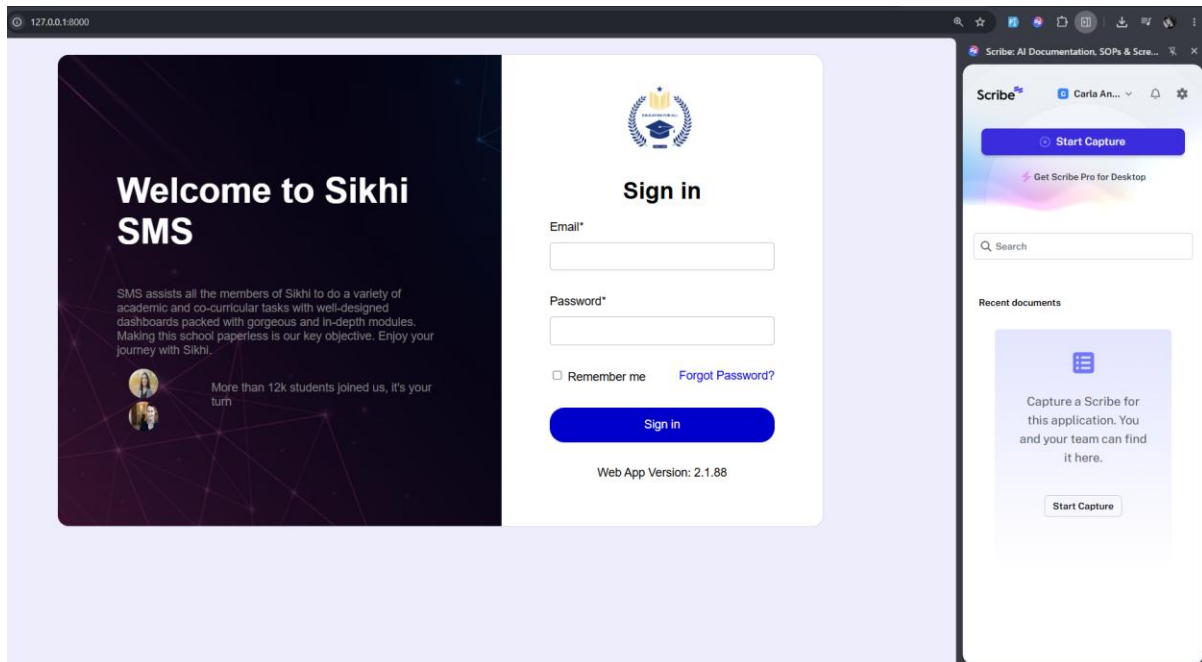
Paso	Acción
1	El sistema registra continuamente la pantalla del usuario (video o frames).
2	Identifica cambios entre frames para detectar acciones relevantes (clics, scrolls, escritura).
3	Rastrea la posición del mouse para inferir interacción (hover, click, drag).
4	Usa visión por computadora para identificar elementos UI (botones, campos, menús).
5	Extrae texto visible en pantalla (labels, inputs, botones).
6	Clasifica acciones como “click”, “input”, “navegación”, “scroll”.
7	Une eventos pequeños en pasos significativos (ej: “abrir menú → seleccionar opción”).
8	Un modelo de lenguaje convierte acciones técnicas en instrucciones humanas entendibles.
9	Produce una guía paso a paso automática (tutorial o documentación)
10	El usuario puede guardar o compartir el flujo como documento o SOP (procedimiento).

2.2.4. Instalación

- 1) Instalar la extensión de SCRIBE en el navegador



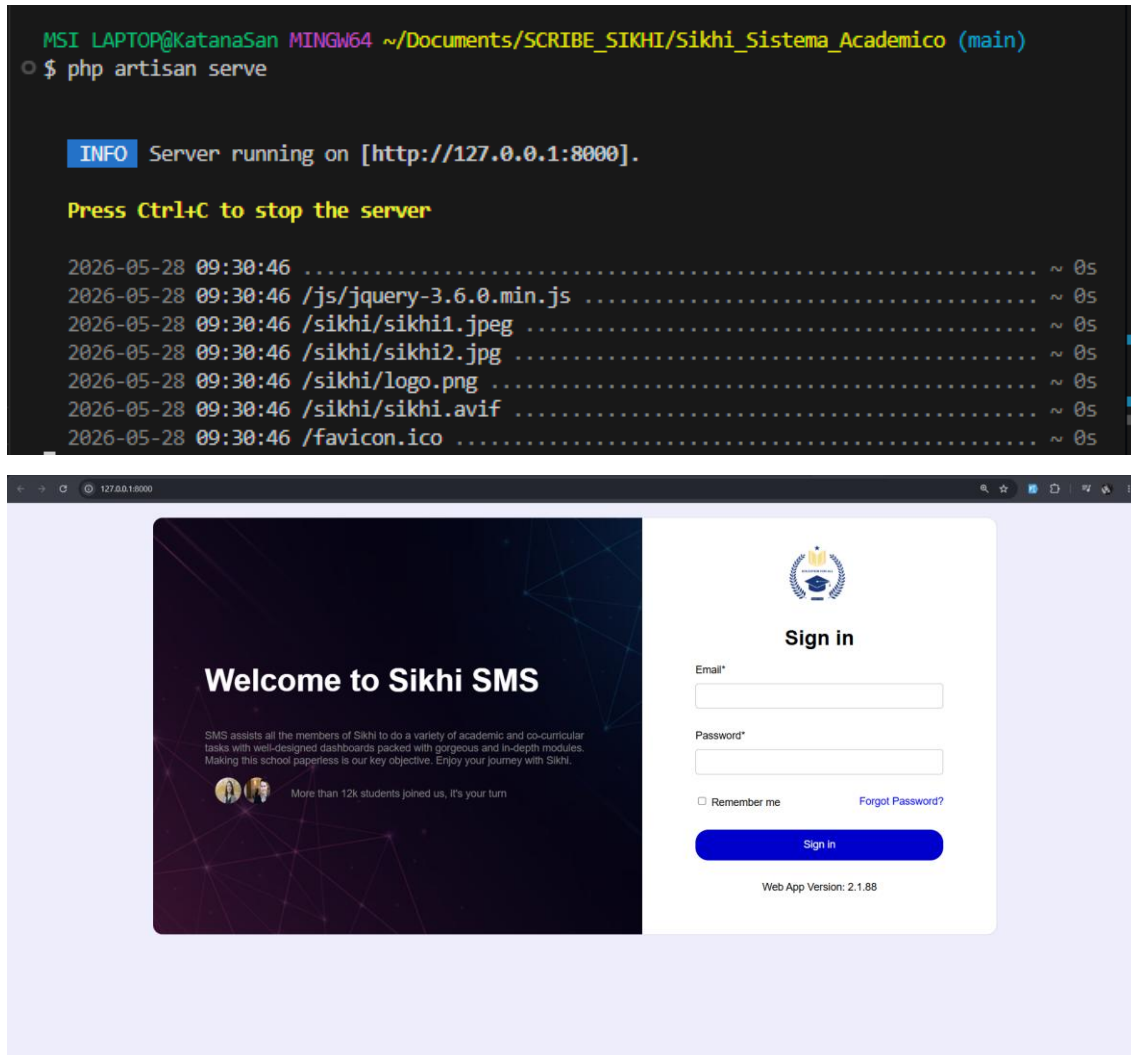
- 2) Añadir a accesos directos la extensión y ya tenemos listo SCRIBE



2.2.5. Integración y Configuración

2.2.5.1. Integración

Se debe tener el sistema iniciado correctamente:

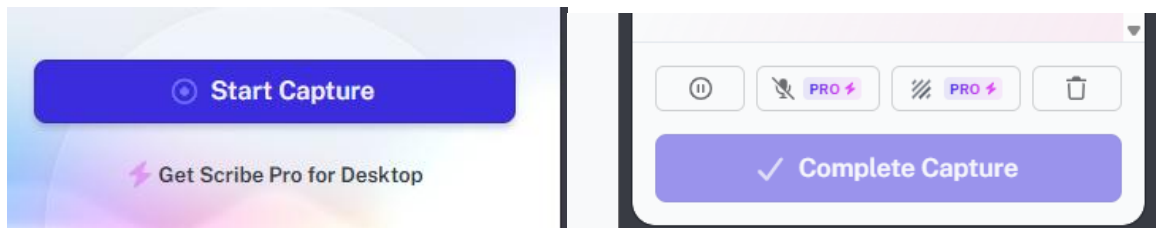


2.2.6. Preview demo pruebas

Scribe es una herramienta que sirve para **crear documentación paso a paso automáticamente** mientras se realiza tareas en una computadora. Por lo cual solamente se pudo:

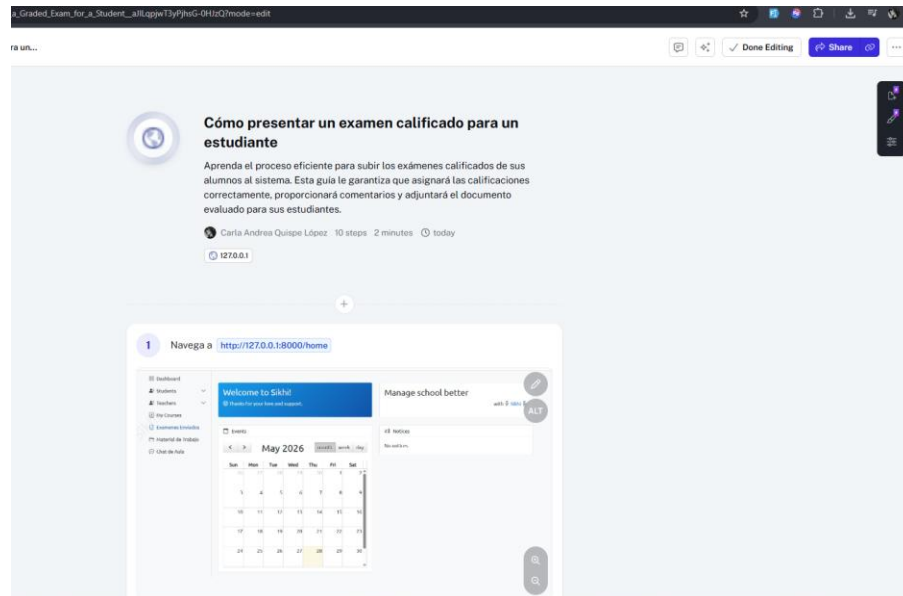
- Registrar lo que haces en pantalla (clics, escritura, navegación)
- Convertir esas acciones en guías automáticas
- Generar tutoriales tipo “cómo hacer” sin escribirlos manualmente
- Ahorrar tiempo documentando procesos repetitivos

Para poder realizar las pruebas es de hacer click en Start Capture para empezar la grabación de la pantalla y COMPLETE CAPTURE para terminar



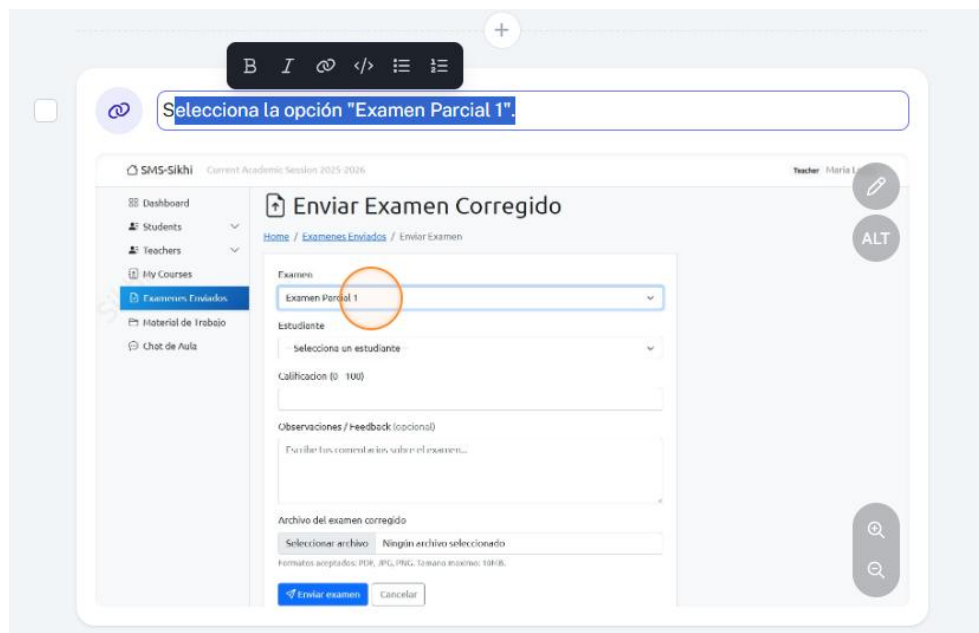
2.2.6.1. *Recorrido del software*

Una vez terminado el recording, se abre el editor de todos los recorridos que se hizo por el sistema.



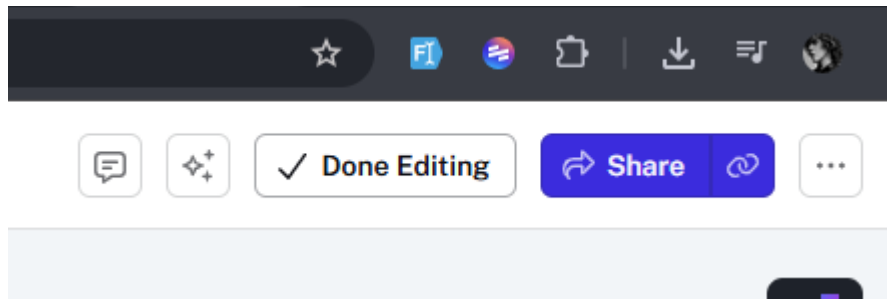
2.2.6.2. *Edición de texto y capturas*

Se puede editar el texto y las capturas



2.2.6.3. Finalización

Al Finalizar se puede compartir un link:



Para vista profesor:

- https://scribeshow.com/viewer/Como_presentar_un_examen_calificado_para_un_estudiante_aJlLqpjwT3yPjhsG-0HJzQ

Para vista alumno:

- https://scribeshow.com/viewer/Como_descargar_los_materiales_del_examen_y_enviar_comentarios_oQdg5SlrThG45qKU84fz9Q

2.2.7. Resultado de la Ejecución de las Pruebas

2.2.7.1. Evaluación de calidad: ¿el manual generado es comprensible para un usuario no técnico?

El manual generado por Scribe cumple de forma satisfactoria con los criterios visuales básicos: las capturas son claras, los elementos interactuados quedan resaltados y los pasos siguen un orden lógico. No obstante, la herramienta no genera contexto de negocio ni cubre situaciones de error, por lo que un usuario que enfrente una situación distinta al flujo exitoso documentado podría quedar sin orientación. En conjunto, el manual es útil como punto de partida pero requiere revisión humana para alcanzar un nivel de calidad adecuado para el usuario final no técnico.

Criterio	Resultado	Observación
Lenguaje accesible para usuarios no técnicos	Parcial	Los pasos generados son directos y sin jerga técnica. Sin embargo, Scribe genera los títulos en ingles por defecto, lo que requirió traducción manual al español.
Capturas de pantalla nítidas en cada paso	Si	Scribe captura fielmente el estado de la pantalla en cada acción realizada, lo que permite al usuario identificar visualmente cada elemento.

Indicadores visuales del elemento a pulsar	Si	La herramienta resalta con un círculo naranja cada botón, enlace o campo sobre el que se actúa, facilitando la orientación del usuario.
Descripción del objetivo de cada Guía	Parcial	Scribe genera una descripción introductoria automática, pero en términos genéricos. Fue necesario reescribirla para que refleje el contexto del sistema Sikhi SMS.
Cobertura de mensajes de error o validación	No	La herramienta no captura mensajes de error ni estados de fallo. Solo registra el flujo exitoso, lo que limita su utilidad ante situaciones inesperadas.
Explicación del propósito de cada acción	No	Scribe describe lo que se hace (por ejemplo, 'haz clic en Guardar') pero no explica por qué se realiza esa acción dentro del proceso.

2.2.7.2. Comparativa: fragmento de la aplicación sin documentar vs. el resultado generado por la IA

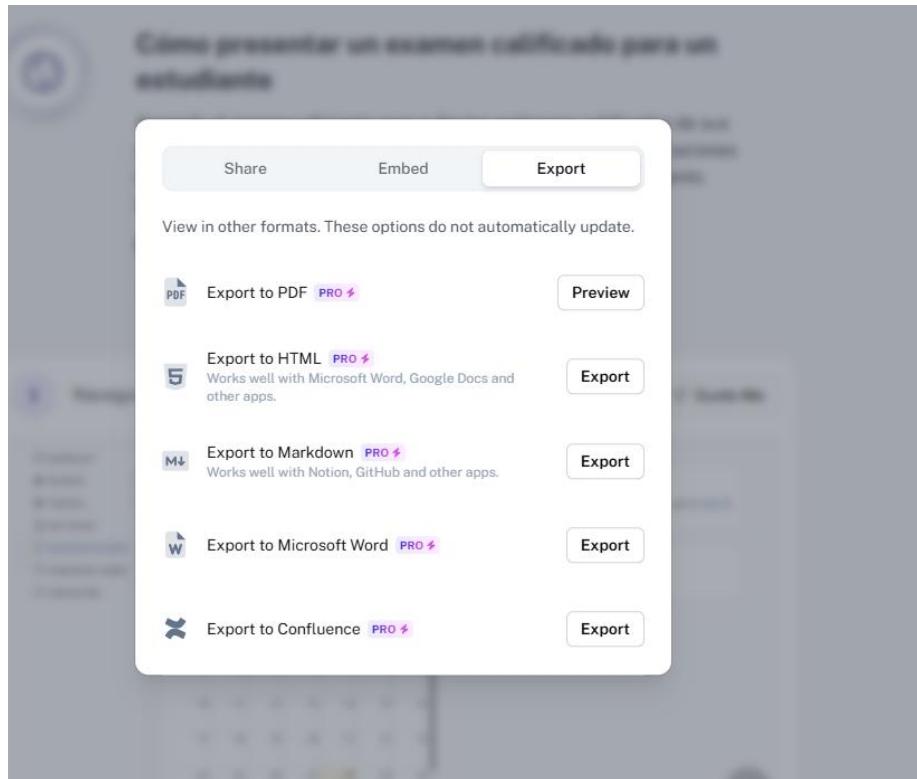
Aspecto	Sin Scribe (manual)	Con Scribe (IA)
Tiempo de elaboración	Entre 30 y 1 hora por flujo documentado	Entre 10 y 15 minutos por flujo, incluyendo edición
Capturas de pantalla	Se toman manualmente, se recortan y se insertan una a una	Se generan de forma automática durante la grabación
Resaltado de elementos	Requiere edición manual con una herramienta de imágenes	Se aplica automáticamente sobre cada elemento interactuado
Texto descriptivo de cada paso	Redactado desde cero por quien elabora el manual	Generado por la IA a partir de las acciones registradas
Consistencia del formato	Depende del criterio y disponibilidad de quien redacta	Uniforme en todas las guías generadas
Actualización ante cambios en la app	Requiere rehacer todo el proceso manualmente	Basta con volver a grabar el flujo modificado

2.2.7.3. ¿Qué partes del manual requirieron corrección o complemento humano?

Parte del manual	Problema detectado en Scribe	Intervención humana realizada
Títulos de cada guía	Generados en ingles con redacción genérica, sin referencia al sistema	Se tradujeron al español y se reformularon para identificar claramente el módulo y el rol involucrado
Descripción introductoria	Texto automático vago, sin contexto del sistema Sikhi SMS	Se reescribió completamente para indicar el objetivo del flujo y a qué tipo de usuario va dirigido
Primer paso de cada guía	Siempre indica 'Navega a 127.0.0.1:8000', una URL local sin significado para el usuario final	Se edito para indicar 'Abre el navegador e ingresa a Sikhi SMS' con la URL correspondiente al entorno real
Pasos irrelevantes incluidos	Scribe registro clics de navegación del navegador y acciones no relacionadas con el flujo documentado	Se eliminaron manualmente los pasos que no forman parte del proceso y podría confundir al usuario
Mensajes de error y validaciones	No fueron capturados en ningún momento por la herramienta	Se agregaron notas aclaratorias al final de cada guía indicando que hacer ante campos vacíos o datos incorrectos
Contexto del negocio	La IA no conoce la lógica del sistema escolar ni los roles de cada usuario	Se incorporaron aclaraciones sobre el rol que debe ejecutar cada flujo y el impacto de cada acción dentro del sistema

2.2.8. Limitaciones encontradas

Al ser una herramienta de paga, esta cuenta con su parte free, por lo cual No se puede exportar a PDF u otros al ser de PAGA:



2.2.9. Conclusión sobre Scribe

En total, se estima que la revisión y corrección manual represento aproximadamente el 30% del tiempo total de elaboración del manual. Las tareas más frecuentes fueron la traducción de textos y la eliminación de pasos irrelevantes. Las más críticas, aunque menos frecuentes, fueron la incorporación de contexto sobre el sistema y la documentación de casos de error, ya que sin ellas el manual podría resultar insuficiente ante situaciones reales de uso.

Esto indica que Scribe es una herramienta eficaz para acelerar la documentación, pero no puede reemplazar completamente al redactor técnico. **Su mayor valor está en la captura automática y el resaltado visual**, mientras que la comprensión del negocio y la cobertura de casos alternativos siguen dependiendo del criterio humano.

3. Reflexión

El análisis comparativo realizado en ambas categorías evidencia que la inteligencia artificial se está consolidando como un apoyo clave en el aseguramiento de calidad del software, tanto en la automatización de pruebas como en la generación de documentación. Sin embargo, también deja claro que su aporte no sustituye el conocimiento humano, sino que lo potencia.

En la automatización de pruebas, herramientas como Qodo muestran una mejora significativa en la detección temprana de errores, la generación de casos de prueba y el análisis contextual del código, reduciendo tiempos de desarrollo y aumentando la cobertura de testing. Aun así, la definición de la lógica de negocio, la validación de escenarios críticos y la decisión final sobre qué pruebas implementar siguen dependiendo del criterio del desarrollador.

De manera similar, en la generación de manuales de usuario, Scribe demuestra una alta eficiencia al capturar automáticamente flujos y convertirlos en documentación visual clara y estructurada. No obstante, la herramienta carece de comprensión del contexto del sistema, del lenguaje del usuario final y de la gestión de casos alternativos o de error, lo que obliga a una intervención humana para asegurar calidad y utilidad real del documento.

En conjunto, ambas herramientas reflejan un mismo patrón: la IA actúa como un acelerador de procesos, mejorando la productividad, reduciendo tareas repetitivas y aumentando la rapidez de entrega, pero la calidad final del resultado depende de la intervención humana para aportar contexto, criterio y validación. Esto confirma que el futuro del aseguramiento de calidad no es la sustitución del rol humano, sino la colaboración entre la inteligencia artificial y el conocimiento profesional especializado.

4. Conclusiones

Tanto Scribe como Qodo demuestran cómo la inteligencia artificial puede optimizar significativamente tareas tradicionales dentro del ciclo de desarrollo y documentación de software, pero también evidencian que su rol es complementario y no sustitutivo del trabajo humano.

En el caso de Scribe, se observa una gran eficiencia en la automatización de la documentación mediante la captura de acciones y generación de guías, reduciendo considerablemente el esfuerzo manual. Sin embargo, sigue siendo necesaria la intervención humana para enriquecer el contenido con contexto, casos especiales y validaciones reales del proceso. De forma similar, Qodo aporta valor al mejorar la calidad del software mediante la detección de errores y la sugerencia de pruebas adicionales con un análisis contextual avanzado, aunque no reemplaza el criterio del desarrollador en la toma de decisiones críticas.

En conjunto, ambas herramientas reflejan una tendencia clara: la IA actúa como un acelerador y asistente inteligente que mejora la eficiencia, la cobertura y la calidad del trabajo, mientras que el conocimiento humano sigue siendo esencial para garantizar precisión, relevancia y comprensión del contexto.